HUMANLINK

Hackathon Project, Product & Technical Documentation

Connecting Every Voice Through Inclusive Technology

Project: HumanLink
Hackathon: MTN Ghana Tɛkyerɛma Pa Hackathon 2026
Primary Domain: Accessibility & Assistive Tools
Core Technology Domain: Speech & Language Technologies
Team: HumanLink
Project Lead: Redeemer Jerrey Kow Williams
Version: 1.0
Date: September 2026

GitHub Repository: https://github.com/HumanLink-Hackathon/HumanLink
GitHub Clone URL: https://github.com/HumanLink-Hackathon/HumanLink.git

---

1. PROJECT OVERVIEW

HumanLink is an inclusive communication technology platform designed to reduce communication barriers between people who communicate differently.

The platform focuses particularly on:

- Deaf and hard-of-hearing users
- People who are more comfortable communicating in Ghanaian languages
- Hearing users who need to communicate with Deaf or hard-of-hearing people
- Educational institutions
- Customer-service environments
- Financial-service environments
- Public institutions and organizations

HumanLink combines speech recognition, language processing, translation, real-time captioning and accessible visual communication into one user-friendly system.

The initial prototype will focus on **Twi and English**, while the architecture will be designed to support additional Ghanaian languages in the future.

---

2. THE HACKATHON CHALLENGE

The MTN Ghana Tɛkyerɛma Pa Hackathon challenges participants to develop inclusive digital solutions that improve communication and accessibility using Ghanaian languages and technology.

HumanLink approaches the challenge from the perspective of:

> **How can technology help people communicate when language or hearing ability creates a barrier?**

Our solution combines Ghanaian-language technology with accessibility.

Rather than building only a translation application, HumanLink focuses on creating a communication bridge between people.

---

3. THE PROBLEM

Communication barriers exist in many everyday environments.

A Deaf person may struggle to communicate with:

- A bank representative
- An MTN customer-service agent
- A lecturer
- A hospital worker
- A government official
- A business employee
- A church representative
- A hearing member of the public

At the same time, many Ghanaians communicate more naturally in Ghanaian languages rather than English.

This creates a second layer of difficulty.

For example:

A customer may speak Twi.

A service representative may understand English.

A Deaf customer may communicate primarily through visual/sign communication.

Without appropriate technology, these people may struggle to understand each other.

HumanLink aims to bridge this gap.

---

4. OUR SOLUTION

HumanLink creates a communication pipeline:

SPEAK → RECOGNIZE → UNDERSTAND → TRANSLATE → DISPLAY → COMMUNICATE

Example:

A user speaks:

> "Mepɛ sɛ meyɛ MTN Mobile Money transaction."

The system can:

1. Capture the speech.
2. Convert speech to text.
3. Identify the language.
4. Process the text.
5. Translate Twi to English.
6. Display the translated text as a live caption.
7. Provide an accessible visual/sign representation where supported.
8. Optionally provide audio output.

The result is a communication experience that does not depend on everyone communicating in exactly the same way.

---

5. CORE MVP

We are not attempting to solve every accessibility and language problem in Ghana during the hackathon.

Our MVP will demonstrate one complete, reliable communication flow.

MVP FLOW

Input

Twi or English speech.

↓

Speech Recognition

Convert speech into text.

↓

Language Processing

Identify and process the recognized language.

↓

Translation

Translate between Twi and English.

↓

Real-Time Captions

Display the recognized or translated speech as large, readable text.

↓

Accessibility Output

Provide visual/sign-language representation for selected supported phrases.

↓

Optional Audio

Provide speech output where appropriate.

---

6. PRIMARY DEMONSTRATION

Our strongest demonstration will be an **inclusive MTN customer-service scenario**.

Scenario

A Deaf customer needs assistance with an MTN service.

The customer and hearing representative cannot communicate easily.

HumanLink is opened.

The hearing representative speaks.

HumanLink processes the communication through:

Speech → Text → Translation → Caption

The Deaf customer sees the communication visually.

Where supported, HumanLink can provide a visual/sign representation for selected phrases.

The Deaf customer can respond through the available communication interface.

This demonstrates that HumanLink is not simply a translator.

It is a **communication bridge**.

---

7. SECONDARY DEMONSTRATION

University Lecture

A lecturer speaks during a lecture.

HumanLink captures the lecturer's speech and generates live captions.

A Deaf or hard-of-hearing student can read the lecture content in real time.

Example:

Lecturer:

> "Today we are discussing artificial intelligence."

HumanLink:

> TODAY WE ARE DISCUSSING ARTIFICIAL INTELLIGENCE.

This demonstrates the educational accessibility use case.

---

8. FINANCIAL INCLUSION

Financial inclusion is an important part of the HumanLink concept.

Many users interact with financial and mobile-money services through customer-service channels.

HumanLink can make these interactions more accessible.

Potential scenarios include:

- Understanding mobile-money instructions
- Communicating with customer-service representatives
- Understanding transaction information
- Asking questions about financial services
- Receiving accessible explanations

IMPORTANT MVP LIMITATION

HumanLink will not claim to perform real MTN Mobile Money transactions unless the team has official authorization and access to the required APIs.

The hackathon prototype will demonstrate an **accessible financial-service communication experience**, not an unauthorized live financial transaction system.

---

9. SIGN-LANGUAGE COMPONENT

Sign language is an important accessibility component of HumanLink.

However, we must be technically honest about the prototype.

The MVP should not claim:

> "HumanLink provides complete Ghanaian Sign Language translation."

Instead, the MVP should provide:

> **Controlled sign-language representations for selected phrases and supported scenarios.**

These may initially be represented using:

- Sign illustrations
- Image sequences
- Short sign videos
- Controlled animations
- A sign-language avatar
- Predefined sign sequences

The long-term objective is a much broader Ghanaian Sign Language technology system.

---

10. INITIAL LANGUAGE SCOPE

MVP Languages

Twi

Primary Ghanaian language for the prototype.

English

Primary cross-language communication language.

Future Languages

The architecture should allow future support for:

- Ga

- Ewe
- Fante
- Dagbani
- And other Ghanaian languages

We should not attempt to implement all of these languages during the MVP unless time and reliable language resources allow it.

---

11. PRODUCT VISION

HumanLink's long-term vision is:

> **A Ghanaian inclusive communication platform where language and disability do not prevent people from accessing information, services and opportunities.**

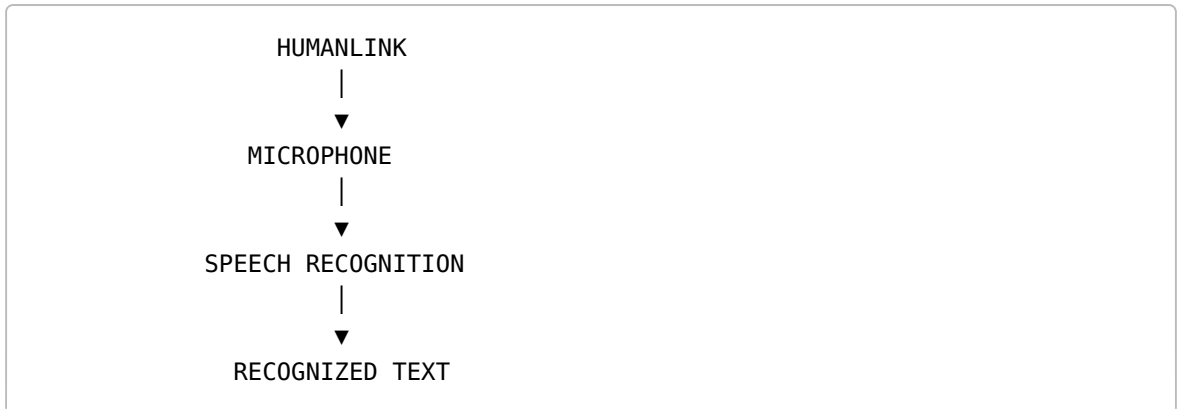
Future versions could support:

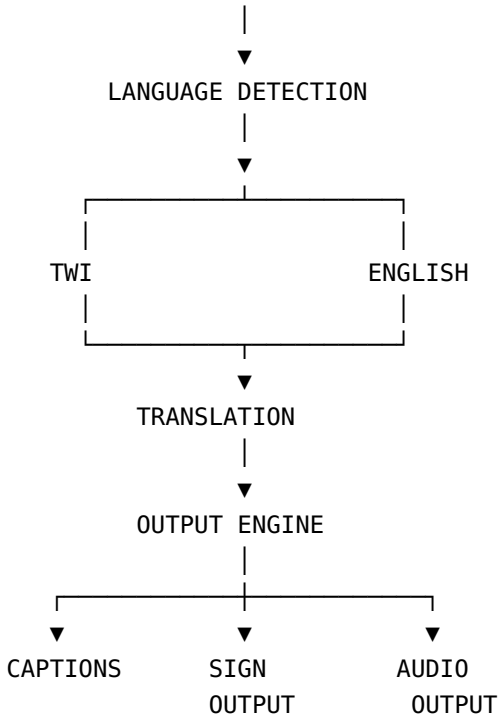
- More Ghanaian languages
- Advanced Ghanaian Sign Language technology
- Mobile applications
- Schools
- Universities
- Hospitals
- Banks
- MTN service centres
- Government offices
- Churches
- Public events
- Customer-service centres
- Emergency communication
- Accessible digital financial services

---

12. SYSTEM ARCHITECTURE

The basic architecture is:





---

13. APPLICATION ARCHITECTURE

Flutter will be the primary application framework.

Technology Stack

Frontend

- Flutter
- Dart

Development

- VS Code
- Android Studio
- Git
- GitHub

Backend

Depending on the final architecture:

- Firebase
- REST API
- Cloud Functions
- Python service where AI/model processing requires it

AI/LANGUAGE SERVICES

Potential components include:

- Speech recognition
- Language detection
- Machine translation
- Natural Language Processing
- Text-to-Speech
- Sign-language processing/representation

The exact AI provider/model will be selected after testing accuracy, latency, cost and licensing.

---

14. IMPORTANT ARCHITECTURAL PRINCIPLE

Flutter is the **application layer**.

The AI does not need to be written entirely inside Flutter.

The architecture can be:

```
Flutter App
     │
     ▼
API / AI Service
     │
     ├── Speech Recognition
     ├── Language Detection
     ├── Translation
     ├── NLP
     └── Sign Processing
     │
     ▼
Flutter App
     │
     ├── Captions
     ├── Visual Output
     ├── Sign Output
     └── Audio Output
```

This allows the team to change AI services later without rebuilding the entire application.

---

15. FLUTTER PROJECT STRUCTURE

The main repository should contain one Flutter application.

We should not create separate Flutter applications for Twi and English.

Recommended structure:

```
HumanLink/
│
├── android/
├── ios/
├── web/
│
├── assets/
│   ├── images/
│   ├── icons/
│   ├── audio/
│   ├── signs/
│   └── translations/
│
├── lib/
│   │
│   ├── ai/
│   │   ├── speech/
│   │   ├── translation/
│   │   ├── language_detection/
│   │   └── sign_language/
│   │
│   ├── core/
│   │   ├── constants/
│   │   ├── theme/
│   │   ├── utils/
│   │   └── accessibility/
│   │
│   ├── features/
│   │   ├── home/
│   │   ├── conversation/
│   │   ├── captions/
│   │   ├── sign_language/
│   │   └── financial_services/
│   │
│   ├── models/
│   │
│   ├── screens/
│   │
│   ├── services/
│   │   ├── api/
│   │   ├── audio/
│   │   └── storage/
│   │
│   └── widgets/
│
├── test/
```

```
│
├── docs/
│   ├── architecture/
│   ├── research/
│   ├── accessibility/
│   ├── api/
│   └── meetings/
│
├── .github/
│   ├── workflows/
│   └── ISSUE_TEMPLATE/
│
├── README.md
├── CONTRIBUTING.md
├── LICENSE
└── pubspec.yaml
```

---

16. USER INTERFACE

HumanLink must be designed as an accessibility-first application.

The interface should prioritize:

- Large readable text
- Large buttons
- High contrast
- Simple navigation
- Clear icons
- Minimal clutter
- Visual feedback
- Caption controls
- Sign-language controls
- Audio controls
- Easy language switching

The interface should work for users who may have different levels of:

- Hearing ability
- Vision
- Technical literacy
- Language proficiency

---

17. PROPOSED MAIN SCREENS

Screen 1 — Welcome

HumanLink logo.

Tagline:

> Connecting Every Voice Through Inclusive Technology.

Buttons:

- Start Communication
- Accessibility Settings

Screen 2 — Select Communication Mode

Options:

- Speech → Text
- Twi → English
- English → Twi
- Live Captions
- Accessibility Mode
- Financial Service Assistance

Screen 3 — Conversation

Main elements:

```
-------------------------------
        HUMANLINK
-------------------------------

Language:
[Twi ▼]

Listening...

🎤

"Mepɛ sɛ meyɛ MTN Mobile
Money transaction."

-------------------------------
```

```
English:

"I want to make an MTN Mobile Money transaction."

--------------------------------

[ Sign ] [ Speak ] [ Replay ]
--------------------------------
```

---

18. LIVE CAPTIONING

The caption system should:

- Display recognized speech quickly
- Keep text large
- Distinguish speaker/output where possible
- Show translation separately
- Allow users to pause captions
- Allow text-size adjustment
- Avoid excessive animation
- Maintain readable contrast

Potential future feature:

Speaker identification.

Example:

```
AGENT:
How can I help you?

CUSTOMER:
I want to check my account.
```

---

19. LANGUAGE TRANSLATION

Translation should be treated as a separate service.

Example:

```
INPUT LANGUAGE:
Twi
```

```
INPUT:
Mepɛ sɛ meyɛ MTN Mobile Money transaction.

PROCESSING

OUTPUT LANGUAGE:
English

OUTPUT:
I want to make an MTN Mobile Money transaction.
```

Translation quality must be tested with fluent speakers.

We should not rely entirely on literal word-for-word translation.

---

20. SPEECH RECOGNITION

Speech recognition should:

1. Request microphone permission.
2. Capture speech.
3. Send/process audio.
4. Receive recognized text.
5. Display text.
6. Pass text to the language-processing layer.

Example:

```
Microphone
    ↓
Audio
    ↓
Speech Recognition
    ↓
"Mepɛ sɛ meyɛ..."
    ↓
Translation
```

---

21. ACCESSIBILITY ENGINE

Accessibility should not be treated as an optional visual feature.

It should influence the whole application.

Possible settings:

```
Text Size
[ Small ] [ Medium ] [ Large ]

High Contrast
[ ON / OFF ]

Captions
[ ON / OFF ]

Sign Support
[ ON / OFF ]

Audio
[ ON / OFF ]

Reduce Motion
[ ON / OFF ]
```

---

22. SIGN OUTPUT

For the MVP, create a controlled library.

Example:

```
Phrase
   ↓
"How can I help you?"
   ↓
Sign representation
   ↓
Image / animation / video
```

The system can map selected phrases to sign resources:

```
assets/signs/
    hello/
    help/
    thank_you/
    mobile_money/
    account/
    transaction/
```

The team should document the source and validation of each sign resource.

---

23. DATA MODEL

Potential models:

```
User
Conversation
Message
Translation
Caption
Language
SignResource
AccessibilitySettings
```

Example:

```
Message
----------------
id
originalText
translatedText
sourceLanguage
targetLanguage
timestamp
audioReference
signReference
```

---

24. BACKEND

A backend should only be introduced where it adds value.

Possible responsibilities:

- API communication
- AI processing
- User/session management
- Storing non-sensitive data
- Sign-resource management
- Analytics during development

Firebase can be used for:

- Authentication if needed
- Firestore
- Storage
- Hosting

- Cloud Functions

However, we should avoid unnecessary backend complexity during the hackathon.

---

25. GITHUB REPOSITORY

HumanLink's official GitHub repository is:

Repository: https://github.com/HumanLink-Hackathon/HumanLink

Clone URL:

```
git clone https://github.com/HumanLink-Hackathon/HumanLink.git
```

To connect an existing local project to the repository:

```
git remote add origin https://github.com/HumanLink-Hackathon/HumanLink.git
```

To verify the configured remote:

```
git remote -v
```

To push the project to GitHub:

```
git add .
git commit -m "chore: initialize HumanLink project"
git branch -M main
git push -u origin main
```

If the remote already exists, update it with:

```
git remote set-url origin https://github.com/HumanLink-Hackathon/
HumanLink.git
```

The repository should contain the Flutter application, documentation, assets, tests and project configuration.

---

26. GITHUB ORGANIZATION

HumanLink is hosted under the GitHub Organization:

```
HumanLink-Hackathon
```

Main repository:

```
HumanLink
```

Official repository link:

```
https://github.com/HumanLink-Hackathon/HumanLink
```

---

27. GITHUB TEAM

Current GitHub usernames:

Member	GitHub
Redeemer Jerrey Kow Williams	`soarmedia`
Pearl Tulasi Mawufemo	Pending
Enock Kyei-Baffour	`Courteous05`
Hannah Aidoo	`faculty-NA`
Able Mwintuma Gambo	`mwintuma`

Pearl's GitHub username should be added when received.

---

28. GITHUB PERMISSIONS

Recommended:

Main Branch

`main`

Production/demo-ready code only.

Development Branch

`develop`

Integration branch for completed features.

Feature Branches

Examples:

```
feature/speech-recognition
feature/translation
feature/live-captions
feature/accessibility-ui
feature/sign-language
feature/financial-services
feature/home-screen
```

Bug fixes:

```
fix/caption-overflow
fix/audio-permission
fix/translation-error
```

---

29. DEVELOPMENT WORKFLOW

No direct pushing to `main`.

Recommended workflow:

```
Issue
  ↓
Feature Branch
  ↓
Development
  ↓
Testing
  ↓
Pull Request
  ↓
Code Review
  ↓
Merge
  ↓
Develop
  ↓
Final Testing
  ↓
Main
```

Every significant feature should have a GitHub Issue.

To push a feature branch:

```
git checkout -b feature/feature-name
git add .
git commit -m "feat: describe the feature"
git push -u origin feature/feature-name
```

After pushing, open a Pull Request on:

```
https://github.com/HumanLink-Hackathon/HumanLink
```

---

30. GITHUB ISSUES

Create issues such as:

```
#1 Initialize Flutter project
#2 Create HumanLink UI
#3 Implement microphone permission
#4 Implement speech recognition
#5 Implement Twi-English translation
#6 Implement live captions
#7 Create accessibility settings
#8 Create sign-language resource system
#9 Create financial-service demo
#10 Backend/API integration
#11 Testing
#12 Demo preparation
```

---

31. DOCUMENTATION

The repository should contain:

```
docs/
├── architecture/
├── research/
├── accessibility/
├── api/
└── meetings/
```

Documentation should include:

- Architecture decisions
- AI research
- Language research
- Accessibility research
- API documentation
- Meeting notes
- Testing results
- Design decisions

This will help prove that HumanLink is a genuine team project.

---

32. TEAM RESPONSIBILITIES

Redeemer Jerrey Kow Williams

Team Lead / Project & Technical Lead

Responsibilities:

- Project direction
- Technical architecture
- GitHub organization
- Repository management
- Flutter architecture
- Integration
- Code review
- Team coordination
- Final technical integration
- Hackathon presentation coordination

Hannah Aidoo

Development / Research Support

Responsibilities can include:

- Flutter development
- Feature implementation
- Language research
- Testing
- Documentation
- User-flow validation

Final assignment should be confirmed with Hannah based on her strengths.

Enock Kyei-Baffour

Backend / AI Engineering

Potential responsibilities:

- Backend services
- API integration
- AI service integration
- Speech/translation experimentation
- Data processing
- Technical testing

Pearl Tulasi Mawufemo

UI/UX & Accessibility

Responsibilities:

- User interface
- User experience
- Accessibility design
- Figma prototypes
- Visual hierarchy
- Accessibility testing
- Caption interface
- Usability testing

Able Mwintuma Gambo

Software Development / Data & Testing

Potential responsibilities:

- Flutter development
- Data structures/models
- Feature implementation
- Testing
- Debugging
- AI/data experimentation

Roles can be adjusted according to each member's actual technical strengths.

---

33. DESIGN SYSTEM

HumanLink should have a consistent visual identity.

Suggested direction:

Primary Feel

- Modern
- Professional
- Accessible
- Friendly
- Trustworthy
- Technology-focused

UI

- Rounded cards
- Large controls
- Clear icons
- High contrast
- Strong typography
- Minimal clutter

The design must prioritize accessibility over decoration.

---

34. SECURITY & PRIVACY

HumanLink may process sensitive communication data.

Therefore:

- Do not store microphone recordings unnecessarily.
- Do not expose personal information in GitHub.
- Do not commit API keys.
- Use `.env` /secret management where appropriate.
- Do not commit Firebase credentials.
- Do not commit private user data.
- Do not upload personal recordings without consent.

Never commit:

```
.env
google-services.json
service-account.json
API keys
private credentials
passwords
```

unless the file is specifically intended to be public and contains no secrets.

---

35. TESTING

Testing must cover:

Functional Testing

- Speech recognition
- Translation
- Captions
- Language selection
- Sign output
- Audio output
- Navigation

Accessibility Testing

- Text size
- Contrast
- Button size
- Screen-reader compatibility where applicable
- Visual feedback
- Caption readability
- Ease of navigation

Language Testing

Test:

- Different accents
- Different speaking speeds
- Common Twi phrases
- Natural sentence structures
- English translations

---

36. PERFORMANCE

The MVP should prioritize:

- Low latency
- Reliable speech recognition
- Fast translation
- Smooth captions
- Minimal crashes
- Simple UI
- Low network usage where possible

A technically simple system that works reliably is better than a complex system that fails during the demo.

---

37. MVP SUCCESS CRITERIA

The MVP will be considered successful if we can demonstrate:

1.

A user speaks Twi.

2.

HumanLink recognizes the speech.

3.

The system produces readable text.

4.

The system translates the text into English.

5.

The English appears as a clear caption.

6.

The system provides a supported visual/sign output for selected phrases.

7.

The interface remains accessible and easy to use.

8.

The complete process can be demonstrated reliably.

---

38. WHAT WE ARE NOT BUILDING

To prevent scope creep, HumanLink will not attempt to build all of the following during the MVP:

- Every Ghanaian language
- Perfect Ghanaian Sign Language translation
- A complete banking application
- A complete MTN Mobile Money platform
- Unauthorized real financial transactions
- A complete customer-service platform
- A new large language model from scratch
- A universal accessibility solution

- Full offline AI unless technically achievable within the project timeline

---

39. RESEARCH REQUIREMENTS

Before final implementation, the team should research:

Language

- Twi speech resources
- Twi-English translation resources
- Ghanaian language NLP resources
- Common Twi expressions
- Translation accuracy

Accessibility

- Deaf/hard-of-hearing communication barriers
- Ghanaian Sign Language
- Accessible UI design
- Captioning standards
- Communication needs in Ghana

Financial Services

- Customer-service communication barriers
- Accessible financial-service interfaces
- MTN service interactions
- Digital financial inclusion

---

40. USER VALIDATION

Where possible, the team should speak with real users or relevant stakeholders.

Potential participants:

- Deaf/hard-of-hearing students
- Hearing students
- Ghanaian-language speakers
- Lecturers
- Customer-service workers
- Accessibility advocates

Ask:

1. What communication problems do you experience?
2. Which language do you normally use?
3. What happens when the other person does not understand your language?

4. Would live captions help?
5. Would visual/sign output help?
6. What would make the interface difficult to use?
7. What would make you trust the system?

User feedback should influence the final prototype.

---

41. DEVELOPMENT PHASES

PHASE 1 — FOUNDATION

- Create GitHub Organization
- Create the HumanLink repository
- Connect the local project to `https://github.com/HumanLink-Hackathon/HumanLink.git`
- Create Flutter project
- Configure branches
- Create README
- Create folder architecture
- Create initial UI
- Configure assets
- Push the initial project to GitHub

PHASE 2 — USER INTERFACE

Build:

- Welcome screen
- Home screen
- Communication screen
- Language selector
- Accessibility settings
- Caption interface

PHASE 3 — SPEECH

Implement:

- Microphone permissions
- Speech capture
- Speech-to-text
- Error handling
- Loading states

PHASE 4 — LANGUAGE

Implement:

- Twi processing
- English processing
- Language selection
- Twi → English translation
- English → Twi translation where reliable

PHASE 5 — CAPTIONS

Implement:

- Live text display
- Translation display
- Caption sizing
- Caption controls
- Visual feedback

PHASE 6 — SIGN SUPPORT

Implement:

- Supported phrase database
- Sign resources
- Sign display
- Sign selection
- Sign output interface

PHASE 7 — FINANCIAL-SERVICE DEMO

Create a controlled demonstration.

Example:

```
Customer:
"I want to check my mobile money balance."

HumanLink:
Speech → Text → Translation → Accessible Output
```

No unauthorized real transaction.

PHASE 8 — TESTING

Test:

- Different speakers
- Different phrases
- Different environments
- Translation accuracy
- Accessibility
- Network conditions
- Application stability

PHASE 9 — DEMO

Prepare:

- Demo script
- Pitch
- Prototype
- Architecture diagram
- Problem statement
- Impact statement
- User story
- Team presentation
- Backup demo

---

42. DEMO SCRIPT

The final demonstration should tell a story rather than simply showing features.

Opening

> "Imagine needing an essential service but the person serving you cannot communicate with you."

Then introduce the problem.

Demonstration

A user speaks Twi.

HumanLink recognizes the speech.

The English translation appears.

Captions appear.

Visual/sign support is demonstrated.

Then show the financial-service scenario.

Closing

> "HumanLink does not ask people to change the way they communicate. It uses technology to help people understand one another."

Final statement:

> **HumanLink — Connecting Every Voice Through Inclusive Technology.**

---

43. PROJECT REPOSITORY README

The GitHub README should contain:

```
HumanLink
Connecting Every Voice Through Inclusive Technology.

About
Problem
Solution
Features
Architecture
Technology Stack
Accessibility
Supported Languages
MVP
Screenshots
Installation
Development
GitHub Repository
Team
Roadmap
License
```

The README should include the official repository link:

```
https://github.com/HumanLink-Hackathon/HumanLink
```

---

44. CONTRIBUTING GUIDELINES

Every contributor should:

1. Clone the repository.
2. Create or select an issue.
3. Create a feature branch.
4. Implement the feature.
5. Test locally.
6. Commit clearly.
7. Push the branch to the HumanLink repository.
8. Open a Pull Request.
9. Request review.
10. Fix review comments.
11. Merge after approval.

Clone command:

```
git clone https://github.com/HumanLink-Hackathon/HumanLink.git
```

Example commit:

```
feat: add live caption interface
```

Other examples:

```
fix: resolve microphone permission issue

docs: update architecture documentation

ui: improve accessibility controls

test: add translation test cases
```

---

45. DEFINITION OF DONE

A feature is not considered complete merely because the code works.

A feature is complete when:

- Code is implemented.
- UI is functional.
- Error handling exists.
- Accessibility has been considered.

- The feature has been tested.
- Documentation is updated.
- GitHub Issue is updated.
- Pull Request is reviewed.
- Code is merged into the appropriate branch.
- The updated code is pushed to the official HumanLink repository.

---

46. PROJECT MANAGEMENT

Use GitHub Projects to track:

```
BACKLOG
   ↓
TODO
   ↓
IN PROGRESS
   ↓
REVIEW
   ↓
TESTING
   ↓
DONE
```

Each task should have:

- Owner
- Description
- Priority
- Deadline
- Status

---

47. PRIORITY SYSTEM

P0 — Critical

Must work for the demo.

Examples:

- Speech recognition
- Translation
- Captions
- Core communication flow

P1 — Important

Strongly improves the prototype.

Examples:

- Sign output
- Accessibility settings
- Financial-service demo

P2 — Nice to Have

Only implement if time allows.

Examples:

- Advanced animations
- User accounts
- Conversation history
- Additional languages

---

48. SCOPE CONTROL

The team must avoid adding features simply because they sound impressive.

Every proposed feature must answer:

> Does this directly improve the core HumanLink communication experience?

If no, it goes into the future roadmap.

The priority is:

Reliable MVP > Many unfinished features

---

49. INTELLECTUAL PROPERTY

The team should maintain records of:

- Original code
- Designs
- Research
- AI models/services used
- Third-party packages
- Images
- Sign-language resources

- Audio
- Videos
- Datasets

Every external resource should have its source and license documented.

---

50. FINAL PROJECT DELIVERABLES

By the end of development, HumanLink should have:

Software

- Working Flutter application
- Speech recognition
- Twi/English translation
- Real-time captions
- Accessibility interface
- Controlled sign-language support
- Financial-service communication demo

Technical Documentation

- System architecture
- API documentation
- Setup instructions
- Testing documentation
- Accessibility documentation

Hackathon Materials

- Pitch deck
- Demo video if required
- Product screenshots
- Architecture diagram
- Problem statement
- Impact statement
- Team information
- Official GitHub repository

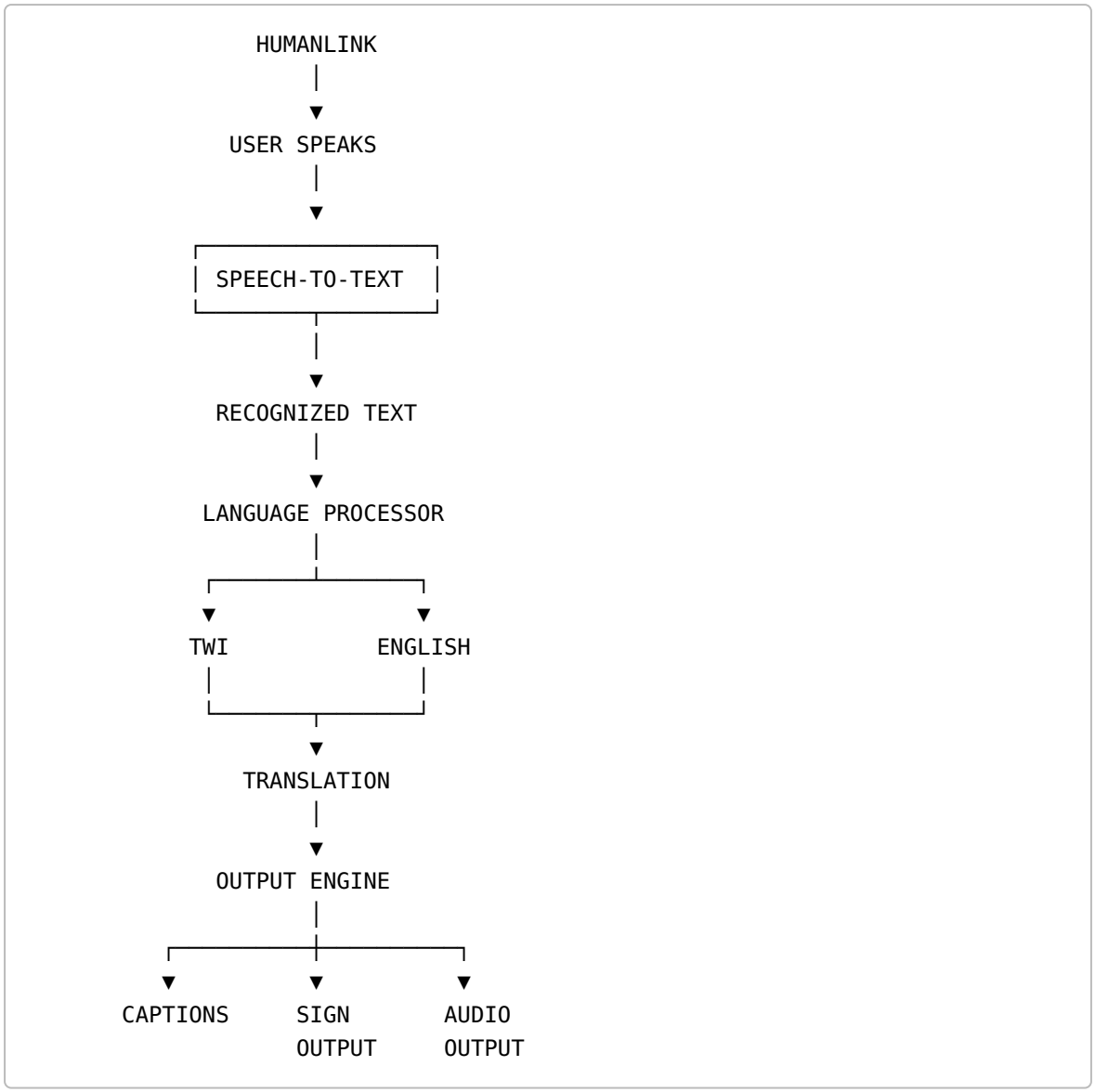
Repository

```
https://github.com/HumanLink-Hackathon/HumanLink
```

---

51. FINAL ARCHITECTURE GOAL

The final MVP should demonstrate:



---

52. HUMANLINK PRODUCT PRINCIPLE

HumanLink is built around one fundamental principle:

> **Technology should adapt to people, not force people to adapt to technology.**

Language should not be a barrier.

Hearing ability should not be a barrier.

Accessibility should not be an afterthought.

HumanLink brings these together into one communication experience.

---

53. FINAL TEAM

HUMANLINK

Redeemer Jerrey Kow Williams
Team Lead

Hannah Aidoo
Team Member

Enock Kyei-Baffour
Team Member

Pearl Tulasi Mawufemo
Team Member

Able Mwintuma Gambo
Team Member

---

54. TEAM TAGLINE

Connecting Every Voice Through Inclusive Technology.

---

55. FINAL PROJECT STATEMENT

HumanLink is not simply a translation tool.

It is an attempt to create a communication bridge between people who speak different languages and people who experience communication differently.

Our first prototype focuses on Twi and English, real-time speech recognition, translation, captions and controlled visual/sign-language support.

The architecture is intentionally designed to grow.

Today:

Twi ↔ English + Accessibility

Tomorrow:

More Ghanaian Languages + Advanced Ghanaian Sign Language + Wider Accessibility

Our vision is simple:

No voice should be left unheard, and no person should be left behind because of how they communicate.

HUMANLINK

Connecting Every Voice Through Inclusive Technology.

Official GitHub Repository:
https://github.com/HumanLink-Hackathon/HumanLink